

MSOL3 - Materials for industrial applications - TP2 : simulation on polycrystalline microstructure

Lucas Lapostolle

Objectives :

- Polycrystalline microstructure random generation.
- Abaqus model generation scripting.

1 Problem description

Metals are composed of a fine crystallographic microstructure in the shape of grains aggregates. Micromechanics is the field of mechanics at the scale of the microstructure, to which this practical session is an introduction. The objectives will be to generate random geometries representative of grains, use anisotropic elastic properties, and perform finite element simulation on the obtained model.

2 Generating microstructures: the Voronoi tessellation

A standard way to give a geometric representation of a polycrystalline is a Voronoi tessellation. A grain is assumed to be a convex shape, i.e. for any point within a grain, the line between these points must be entirely contained within the grain, and not intersect with the boundaries. A Voronoi tessellation achieves this by :

- Initially representing a grain by a unique point : a “seed”.
- With multiple seeds and a uniform grain growth hypothesis, a grain is composed of all the point in space that are closer to its seed than any other.

The random aspect of the microstructure can be achieved by randomly selecting the coordinates of the seed of each grain. Different random Voronoi tessellations are illustrated in Figure 1.

Task : Make a python code to generate your own random voronoi tessellations.

- Use a random generator to create N point of coordinates (x, y) such that $x \in [0, L_x]$ and $y \in [0, L_y]$. Let us consider $L_x = L_y = 1$.
- From the library `scipy.spatial` import `Voronoi`, `voronoi_plot_2d`. To better understand these commands, use the documentation.

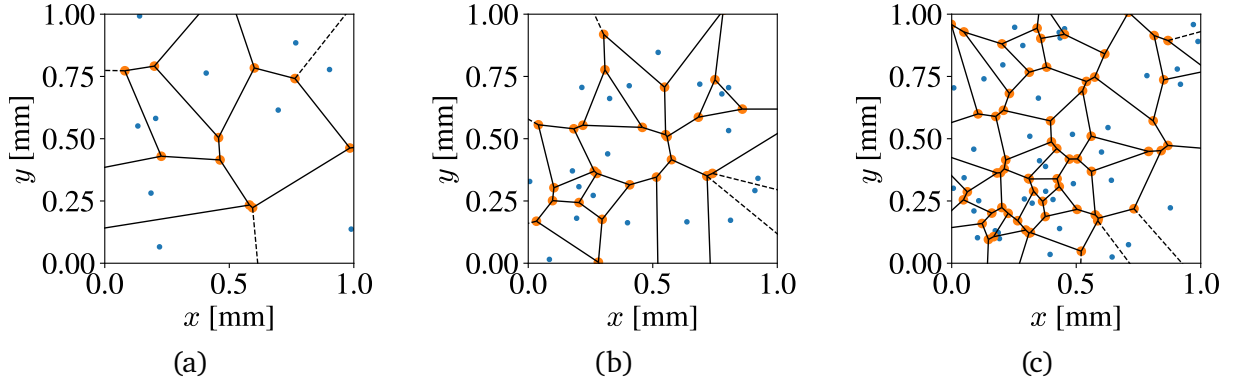


Figure 1: Examples of Voronoi tessellation for different number of grains. Blue points are the seeds, orange points are the vertices of the grains. (a) 10 grains. (b) 20 grains. (c) 40 grains.

- *Voronoi* takes a numpy array $(N, 2)$ containing the coordinates of the seeds. In the following, this array will be referred to as *points*.
- To plot the Voronoi tessellations as in Figure 1, use `voronoi_plot_2d(Voronoi(points))`.

Problem: in this representation, although grains in the center are well represented (finite boundaries), grains close to the edges of the domain can have infinite boundaries, due to the absence of other seeds around them. This can be a problem if we want the boundaries of the grains to be in the $[0, L_x] \times [0, L_y]$ 2D plane.

Task : Perform the same Voronoi tessellation, but **add mirror seeds** with respect to the lines $(x = 0, y)$, $(x = L_x, y)$, $(x, y = 0)$ and $(x, y = L_y)$.

This will ensure to have vertices placed in the corner of the $[0, L_x] \times [0, L_y]$ 2D plane.

To prepare the work to be done in ABAQUS, we need to practise more with the Voronoi tessellation.

Task : Using the outputs of the *Voronoi* method, plot lines delimiting the contour of each grain whose seed is within $[0, L_x] \times [0, L_y]$.

3 Preliminary simulations

In this section we will manually create an ABAQUS model of a simple microstructure.

1. Download the python file *rotate_tensor.py* for Moodle.

Task : Use the previous section to generate a Voronoi tessellation of 3 grains. Manually retrieve the coordinates of the relevant vertices and use them to partition a $1 \text{ mm} \times 1 \text{ mm}$.

$\mathbb{C}_{1111}$ [GPa]	$\mathbb{C}_{1122}$ [GPa]	$\mathbb{C}_{1212}$ [GPa]
106.75	60.41	28.34

Table 1: Orthotropic elastic properties.

Even though the material is isotropic at the microscopic scale, grains are often anisotropic. In the following, we will consider orthotropic aluminum grains, with elastic properties described in Table 1.

In matrix form, the stiffness tensor (in Mandel's notation) is the following :

$$\mathbf{C} = \begin{pmatrix} \mathbb{C}_{1111} & \mathbb{C}_{1122} & \mathbb{C}_{1122} & 0 & 0 & 0 \\ & \mathbb{C}_{1111} & \mathbb{C}_{1122} & 0 & 0 & 0 \\ & & \mathbb{C}_{1111} & 0 & 0 & 0 \\ & & & 2\mathbb{C}_{1212} & 0 & 0 \\ & & & & 2\mathbb{C}_{1212} & 0 \\ & & & & & 2\mathbb{C}_{1212} \end{pmatrix} \quad (1)$$

The anisotropy is enforced by having $2\mathbb{C}_{1212} \neq \mathbb{C}_{1111} - \mathbb{C}_{1122}$ (the behaviour would otherwise be isotropic).

Because of the anisotropic nature of the material, the components of $\mathbf{C}$ can thus be different depending on a rotation angle θ (rotation around an axis z), corresponding to a rotation matrix $\mathbf{P}$. Rotating a stiffness 4th order tensor is more complex than rotating a vector or a 2nd order tensor (matrix). It can be achieved using (in implicit summation notation) :

$$\mathbb{C}'_{ijkl} = \mathbf{P}_{im}\mathbf{P}_{jn}\mathbf{P}_{ko}\mathbf{P}_{lp}\mathbb{C}_{mnop} \quad (2)$$

Task : Use the function in `rotate_stiffness_tensor` in `rotate_tensor.py` to obtain the elastic constant of the rotated stiffness tensor by an angle θ (rad) of your choosing.

The function yields the rotated stiffness tensor components in the order needed by ABAQUS.

2. To define an anisotropic material in ABAQUS, in **Elastic**, change **Isotropic** to **Anisotropic** and fill the corresponding components.

For the rest of the modeling :

- Apply displacement of $u_x = -0.25$ mm and $u_x = 0.25$ mm on left and right sides respectively.
- Apply YSYM boundary conditions on top and bottom sides.
- Mesh the model as you see fit, using plane strain elements.

Task : Run a simulation. Does the mechanical fields reflect the material heterogeneity ?

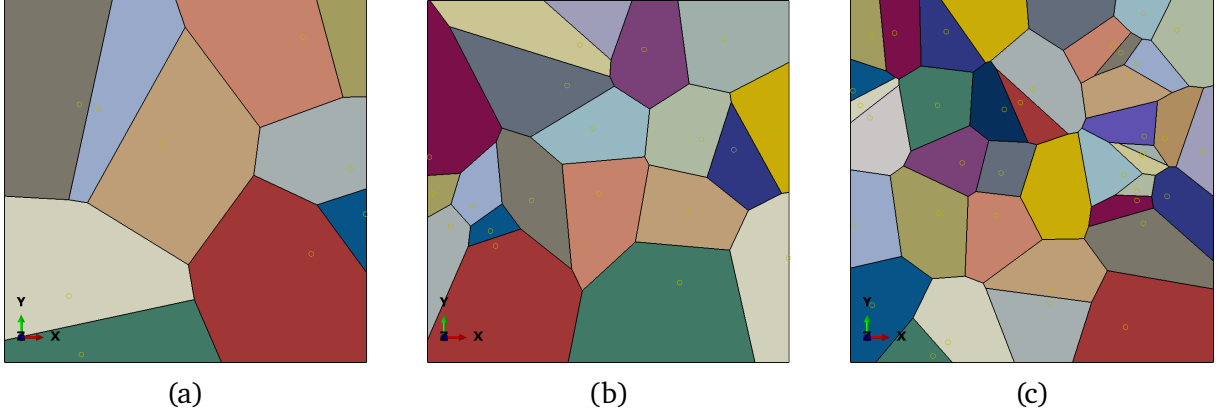


Figure 2: ABAQUS models with random Voronoi tessellations. (a) 10 grains. (b) 20 grains. (c) 40 grains.

4 Random generation of microstructures

The final part is to automatically generate random microstructure in ABAQUS and run simulations. For a large number of grains, it is no longer feasible to do it by hand, and one must resort to ABAQUS scripting.

The creation of an ABAQUS model can be entirely scripted in Python provided one knows the geometry in advance, to achieve results such as in Figure 2. For that, your task will be to complete sections of the code provided in *create_abaqus_voronoi_mmk.py*. Sections to be modified in the code are indicated by comments, anything else must be left as is.

Task 1: In the dedicated section of the code, create an $(N, 2)$ array containing the coordinates of the seeds of the Voronoi tessellation, generated randomly, following the procedure of Section 2. **Do not change the names of the variables.**

For the next task, we will use the *Line* command to draw lines in a sketch to delimitate a grain. The command is be used this way :

```
mdb.models['Model-1'].sketches['__profile__'].
Line(point1=(x1,y1),
point2=(x2,y2))
```

where $x1, y1, x2, y2$ must be replaced by the coordinates of the endpoints of the line.

The *Voronoi* method from *scipy.spatial* contains valuable information, that are readily available in the coding section :

- *vert* contains an $(M, 2)$ array of the coordinates of all the vertices of the Voronoi tessellation.
- *grain* contains N lists of the indices of the vertices forming a grain, in sequence to form a contour.

Task 2: Correctly use the *Line* command, the informations in *vert* and *grain* to draw a line between two vertices. The index k goes through the index of the current *grain* list. As the k index is incremented, your command should make it so that the loop closes the grain.

Next, the correct materials corresponding to the grain orientations must be created, as well as the section, which must then be applied to the correct region of the model.

3. The command to create a material is :

```
mdb.models['Model-1'].Material(name='insert_material_name')
```

Be mindful to specify an incrementing name in order not to name the same material twice.

4. The command to specify anisotropic elastic properties is :

```
mdb.models['Model-1'].materials['insert_material_name'].  
Elastic(table=(tuple_containing_Cijkl, ), type=ANISOTROPIC)
```

Anisotropic elastic properties must be provided in a certain order, fortunately already provided by the function *rotate_stiffness_tensor*.

5. The command to create the section is :

```
mdb.models['Model-1'].HomogeneousSolidSection(  
    material='insert_material_name',  
    name='insert_section_name', thickness=None)
```

Be mindful to specify an incrementing name in order not to name the same section twice, and not to use the same material for different section.

6. To assign the section to a specific area :

```
mdb.models['Model-1'].parts['Part-1'].SectionAssignment(offset=0.0,  
offsetField='', offsetType=MIDDLE_SURFACE, region=Region(  
faces=mdb.models['Model-1'].parts['Part-1'].faces.findAt(((x, y, z),)))
```

Correctly replace the name of the section, and the coordinate x, y, z of a point laying inside the face to be assigned the section (the seed of the current grain might be good choice).

Task 3: Use the previous commands to create the material, the section and assign it to a region. The i index loops over the seeds.

In order to make the model generation fully automatic, special care must be given to how the boundary conditions are selected. Indeed, we will need to apply boundary conditions to the edges of the representative volume element (RVE). These edges are however divided because of the grains geometry. While it could still be not too much cumbersome to select them manually, we will try to make the selection automatic. This procedure will follow the following sequence : create a set containing all the edges for a given boundary condition → creating the boundary condition from this set.

The command to create a set based on geometric edges is :

```
mdb.models['Model-1'].parts['Part-1'].Set(  
    edges=geometric_selection, name='name_of_the_set')
```

We now need to replace the “geometric_selection” variable by the desired edges. To that end, the method **findAt**, is particularly useful, because it allows the selection of a given entity when provided coordinates of a point lying on this geometry. For example :

```
mdb.models['Model-1'].parts['Part-1'].edges.findAt(((0.2,0.6,0),))
```

selects the edge located at coordinates $x = 0.2$ mm, $y = 0.6$ mm and $z = 0$ mm. If no edges are found at this location, nothing is selected.

These geometric entities **can be summed** :

```
edges_sum=mdb.models['Model-1'].parts['Part-1'].edges.  
findAt(((0.2,0.6,0),))+mdb.models['Model-1'].parts['Part-1'].  
.findAt(((0.3,0.1,0),))
```

corresponds to a selection of 2 edges at the given coordinates.

Eventually, creating a boundary conditions requires the following command, in the case of an imposed displacement boundary condition :

```
mdb.models['Model-1'].DisplacementBC(amplitude=UNSET,  
createStepName='Step-1', distributionType=UNIFORM, fieldName='',  
fixed=OFF, localCsys=None, name='BC-1', region=mdb.models['Model-1']  
.rootAssembly.instances['Part-1-1'].sets['set_name'], u1=UNSET,  
u2=value_to_set, ur3=UNSET)
```

with *set_name* the name of the set where the boundary condition needs to be applied. Here the *u2* is assigned a displacement, while the other degrees of freedom are left free.

The command to create a symmetry boundary condition is (here for an XSYM boundary condition):

```
mdb.models['Model-1'].XsymmBC(createStepName='Step-1', localCsys=None,  
name='BC-2', region=mdb.models['Model-1'].rootAssembly.  
instances['Part-1-1'].sets['set_name'])
```

Task 4: modify the code to apply the same boundary conditions as in Section 3. You need to detect which edges you are included in the various boundary conditions, use them to create a set, and give that set to a command creating the appropriate boundary condition.

The rest of the code takes care of meshing and job creation, which is ready to be submitted.

Task: Run simulation with a (reasonably) increasing number of grains. Where seems to be located stress concentration ?

For a finer mesh, look in the code lines to find the command controlling the mesh size, and adjust if needed.

5 Modify the ABAQUS *Input* file

One last thing remains. By default, ABAQUS has given us *plane stress* quadr and tri elements, whereas we would prefer *plane strain* elements¹. A direct solution is to modify the ABAQUS **input .inp** file.

The previous script, or the interface, commands the preprocessor, i.e. the piece of software preparing the model. This preprocessor then creates an input file (automatically when submitting a job), which is the main file containing the mesh, elements, boundary conditions and analysis information passed onto the solver.

Task: The *Write Input* button from the Job manager will create the input file in the work directory. Open this input file, look for 'CPS4R' and replace it with 'CPE4'. Same for eventual 'CPS3' with 'CPE3'.

Task 5: Add code lines to automatically open in the .inp file, go through it, and make the element type replacements.

In the .inp file, a line is commented when starting with **

Do not leave any blank line in the .inp file. If a line should be empty, make it start with **

Finally, the new input file must be imported into the interface to submit the job from there : **File** → **Import** → **Model** → select the new input file.

6 Influence of anisotropy

A way to define the anisotropy of a material is the **Zener coefficient** Z :

$$Z = \frac{2C_{1212}}{C_{1111} - C_{1122}}. \quad (3)$$

Isotropy thus corresponds to $Z = 1$.

Task : for a given microstructure with 20 grains, compare the results for different Zener coefficients, from 1 to 5.

¹Neither are rigorously correct, plane strain being the least incorrect.